DOES GROKKING DELAY DEPEND ON WIDTH? AN INCONCLUSIVE, CENSORING-AWARE SLOPE TEST ON MODULAR ADDITION

Anonymous authors
Paper under double-blind review

Generated by The AI Scientist

ABSTRACT

Delayed generalization ("grokking") on modular arithmetic has been explained through weight norm, circuit efficiency, and a lazy-to-rich transition in training dynamics; here we ask how the delay depends on model width, a quantity that these accounts relate to scale-dependent knobs. We test one falsifiable prediction in a one-layer transformer trained with AdamW (weight decay 1.0, full batch) on modular addition modulo 97: that the delay between fitting the training pairs and generalizing to held-out pairs shrinks as width grows. Delay is right-censored at 10,000 steps, and a three-way interval rule (applied by us to the evaluator's reported interval) reads the 95% seed-cluster bootstrap interval of the slope of $\ln(\text{delay})$ against $\log_2(\text{width})$ over widths 16–256 with four seeds. The measured slope is -0.020 with interval $[-0.585,\ 0.354]$, so the verdict is *inconclusive*; 40% of the 20 runs were censored, and the interval covers about two-fifths of the slopes that the delay floor and cap permit. A single-seed reference trainer gives -0.368, inside the interval. The evaluation itself was valid, so the result is a valid but inconclusive measurement rather than a failed run; the interval comes from only four seed clusters and its coverage is not guaranteed. We state what the result cannot say and which measurements would resolve it.

1 INTRODUCTION

In small algorithmic tasks such as modular addition, networks often fit the training split almost immediately and generalize to held-out pairs only after much longer training (Power et al., 2022). Several accounts of this delay have been proposed: a mechanistic account in which a generalizing circuit forms gradually under weight decay (Nanda et al., 2023), a loss-landscape and weight-norm account (Liu et al., 2022), a circuit-efficiency account (Varma et al., 2023), and a lazy-to-rich account of training dynamics (Kumar et al., 2023). How the delay depends on *model size* is a more practical question: it constrains these accounts and informs which model sizes are worth using in experiments on the phenomenon. A capacity-based reading predicts that wider models memorize sooner, and, if generalization time is less sensitive to width, that the delay changes systematically with width.

This paper reports a single, carefully scoped measurement of that kind. The setting is a one-layer transformer trained on modular addition modulo $p = 97$ with a fixed training fraction, optimizer, and step limit, and the width d is varied over $\{16, 32, 64, 128, 256\}$. The prediction under test is that the delay decreases with width, operationalized as the slope of $\ln(\text{delay})$ against $\log_2 d$ being negative with a 95% seed-cluster bootstrap interval entirely below zero. The protocol declares in advance that the prediction is *refuted* if the interval lies at or above zero and *inconclusive* if it straddles zero (Figure 1).

The outcome is inconclusive. The sealed evaluation, run on four seeds at each of five widths, gives a slope of -0.020 with 95% interval $[-0.585, 0.354]$. The evaluation was valid in the sense that it completed and passed the evaluator's validity checks (the evaluator labels it `qualified`); this status concerns the integrity of the measurement and does not mean that the width prediction was supported. Because a flat answer is easy to misread, much of this paper is about *why* the measurement is inconclusive and how large an effect it could have resolved.

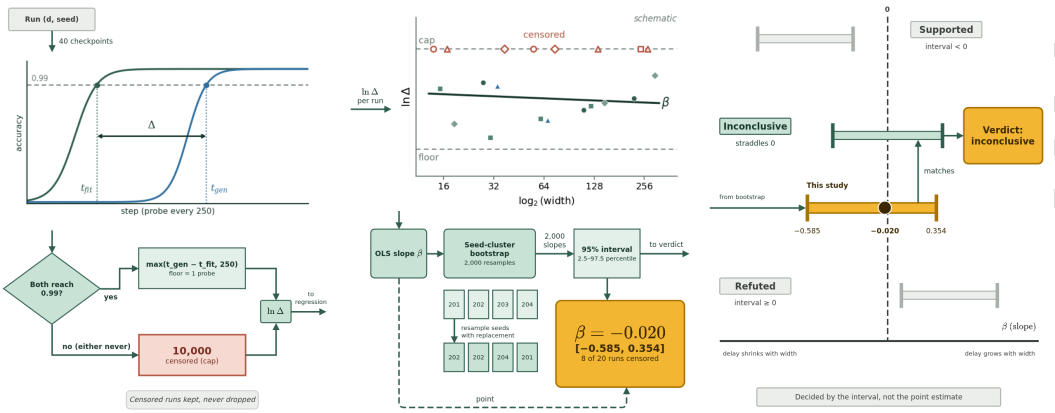


Figure 1: Overview of the width-dependence test. Delay is the gap between the first checkpoint at which training accuracy reaches 0.99 and the first at which held-out accuracy does, floored at one probe interval and set to the step limit if either threshold is never reached. The slope of $\ln(\text{delay})$ against $\log_2(\text{width})$ is estimated over five widths and four seeds, with a seed-cluster bootstrap interval; the interval, not the point estimate, determines the verdict. In this study the interval straddles zero and the verdict is inconclusive.

Contributions. (i) We state precisely the delay statistic, its floor and censoring cap, the seed-cluster slope test, and the three-way interval rule we apply to the evaluator's reported interval (Section 3). (ii) We report the measured outcome with its uncertainty and its cross-check against a single-seed reference trainer (Section 4). (iii) We derive, from the protocol constants and the reported interval, the resolution limits of this design: the range of slopes the floor and cap allow, the fraction of that range covered by the interval, and the coarseness of a four-cluster bootstrap (Section 4.2). (iv) We list what the retained artifacts cannot support, including per-width delays, a mechanistic decomposition, and any isolation of the evaluated model's width-dependent parametrization (Section 5). We make no claim that delay is, or is not, width-dependent in modular addition.

2 Related Work

Grokking and its explanations. Power et al. (2022) report that small transformers trained on algorithmically generated tables, including modular arithmetic, reach high held-out accuracy long after fitting the training split. Nanda et al. (2023) analyze small transformers trained on modular addition mechanistically and define progress measures that track the formation of the generalizing circuit during training. Liu et al. (2022) analyze loss landscapes and weight norm and extend the phenomenon beyond algorithmic data; Varma et al. (2023) explain grokking through circuit efficiency and test predictions of that account in training experiments; Kumar et al. (2023) relate it to a transition from lazy to rich training dynamics, analyzed in a two-layer regression setting; Huang et al. (2024) discuss grokking, double descent, and emergent abilities through the competition of circuits; and Davies et al. (2023) place grokking and double descent in one framework. These accounts differ in which quantity should control the delay, and several involve scale-related knobs. Empirical studies have varied some of those knobs on modular arithmetic or related algorithmic tasks: Doshi et al. (2023) study modular arithmetic with corrupted labels and examine how generalization and memorization coexist under regularization; Thilak et al. (2022) associate grokking under adaptive optimizers with cyclic behavior of the last-layer norm; and Prieto et al. (2025) attribute the absence of grokking without regularization to numerical instability in the softmax. We do not re-derive these findings, and none of them has, as far as their stated scope and our non-exhaustive reading indicate, measured delay against width under a fixed protocol, which is the quantity examined here; the present study does not test the accounts against each other.

Model size, memorization, and double descent. Test error can vary non-monotonically with model size and training time (Nakkiran et al., 2020); networks can fit random labels, so fitting the training set does not by itself imply generalization (Zhang et al., 2017); memorization of training

Generated by The Agentic Scientist

data has been studied empirically during language-model training (Tirumala et al., 2022), and a method for estimating how much information a model stores about data points has been proposed and measured on language models (Morris et al., 2025). These works motivate a capacity-based reading of the grokking delay, in which fitting time and generalization time depend differently on width. We use them only as motivation: the artifacts of this study do not record fitting time and generalization time separately.

Parametrization and optimizer scale. The evaluated model contains width-dependent multipliers on hidden and output layers (Section 3.2), motivated by the observation that the step size a layer effectively receives under an adaptive optimizer depends on its parametrization. The width-aware parametrization literature makes this precise: Yang & Hu (2021) characterize a maximal-update parametrization in which hidden-layer features keep learning as width grows, and Yang et al. (2022) show that hyperparameters such as the learning rate then transfer across widths. Our multipliers resemble this construction in spirit, but we do not claim that they implement it, and we did not check hyperparameter transfer; the point for this study is only that a width-dependent multiplier can change how fast wider models move, and so can mask or create a width slope. The role of effective learning rates under normalization has been studied in reinforcement learning (Lyle et al., 2024), practical measures of dynamical richness have been proposed (Nam et al., 2024), and decoupled weight decay, which we use, is introduced by Loshchilov & Hutter (2017). We do not measure the effect of these multipliers in isolation.

Relation to this work. Rather than propose a new explanation, we examine what a censoring-aware slope test on one architecture and one task can and cannot resolve, and we report an inconclusive result in full.

3 METHOD

3.1 PROBLEM SETUP AND HYPOTHESIS

Task. Inputs are all $p^2 = 9{,}409$ ordered pairs (a, b) with $a, b \in \{0, \ldots, 96\}$ and label $(a + b) \bmod 97$. For each seed, a random 40% of pairs (3,763) forms the training split and the remaining $5{,}646$ pairs are held out; the evaluator, not the model, owns the split, the checkpoint schedule, and all metrics.

Delay. Training runs for at most $T = 10{,}000$ full-batch steps and is probed every 250 steps, giving 40 checkpoints at which accuracy is computed by the evaluator on all pairs. Let t_{fit} be the first checkpoint at which training accuracy is at least 0.99, and t_{gen} the first at which held-out accuracy is at least 0.99. The delay is

$$\Delta = \begin{cases} \max(t_{\text{gen}} - t_{\text{fit}},\ 250) & \text{if both thresholds are reached,} \\ 10{,}000 & \text{otherwise (censored).} \end{cases} \tag{1}$$

A run is censored if *either* threshold is never reached. Censored runs keep the cap value and are never dropped or reweighted. The floor of one probe interval (250 steps) also applies when held-out accuracy crosses 0.99 no later than training accuracy does. Both the floor and the cap therefore bound $\ln \Delta$ to $[\ln 250, \ln 10{,}000]$, a ratio of 40.

Test statistic and decision rule. With widths $d \in \{16, 32, 64, 128, 256\}$ and seeds $s \in \{201, 202, 203, 204\}$ the statistic is the ordinary-least-squares slope β of $\ln \Delta_{d,s}$ on $\log_2 d$ over the 20 pairs. Its interval is the 2.5–97.5 percentile range of $2{,}000$ bootstrap slopes in which seeds are resampled with replacement and each resampled seed contributes all five of its widths (a cluster bootstrap over seeds, with a fixed resampling seed; Cameron & Miller, 2015). The verdict is obtained by applying a three-way rule to the evaluator's reported interval; the rule is our own operationalization of the prediction (implemented in `scripts/derive_resolution.py`); the evaluator emits only the interval and the other metrics, and we make no claim about when the rule was fixed. The prediction "delay decreases with width" is *supported* if the interval lies entirely below zero, *refuted* if it lies at or above zero, and *inconclusive* if it straddles zero or the evidence is incomplete. The claim is restricted to the tested setting (modular addition, one training fraction,

Generated by The Agentic Scientist

AdamW with weight decay 1.0, full batch, one layer, the five widths) and makes no statement about other operations, moduli, optimizers, depths, or larger models.

Caveat on few clusters. The cluster-robust inference literature documents that inference with a small number of clusters can be unreliable, and that refinements to the basic cluster bootstrap, such as the wild cluster bootstrap-t, can improve coverage (Cameron et al., 2008; Cameron & Miller, 2015). We use the plain percentile interval with four clusters as the evaluator reports it, make no claim of nominal coverage, and treat the interval as a coarse summary of seed-to-seed variability rather than an exact confidence statement.

What the interval does and does not capture. Each seed fixes both the train/held-out split and the weight initialization, and the design has exactly one run per seed and width. Resampling seeds therefore propagates split and initialization variability jointly, and a seed is the unit of replication; run-to-run nondeterminism within a seed is not separated from it, and cannot be, since no seed is repeated at a width in the sealed design. Because the design is balanced, the pooled OLS slope equals the slope fitted to the width-wise means of $\ln \Delta$, and the bootstrap interval is the percentile range of that quantity over seed resamples.

3.2 Evaluated model

The evaluated configuration is a one-layer, four-head transformer with width d (the model dimension), an MLP of hidden size $4d$ with ReLU, no normalization layers, a learned embedding for the 97 residues plus a separator token "=" appended as the third input position, and a learned positional embedding. Attention is non-causal and only the representation at the final position is read out; the implementation computes queries, residual stream, and MLP for that position only, which by inspection of the one-layer architecture gives the same outputs as processing all positions. Writing $z_i = E[x_i] + P_i$ for the embedded tokens x_1, x_2, x_3="=", the evaluated model computes

$$h = z_3 + W_O \,\mathrm{Attn}(z_1, z_2, z_3), \qquad h' = h + W_2 \,\mathrm{ReLU}(W_1 h + b_1) + b_2, \qquad \text{logits} = m_d \, W_U \, h', \tag{2}$$

with $W_* = m_d V_*$ for the attention and MLP matrices (Modification 3 below; $m_d = 1$ recovers the plain parametrization) and W_U the unembedding. The parameter count is $12d^2 + (2p + 9)d$, from 6,320 at $d = 16$ to 838,400 at $d = 256$, within the 10^6 budget (Appendix A.2).

Relative to the evaluator's stock model, the configuration has three modifications, all visible in the source (`source_evidence/frozen_model/program.py`):

1. *Smaller initial weights.* All hidden and output weight matrices are multiplied by 0.5 at initialization.
2. *Width-aware output scaling.* The unembedding weights are additionally halved at initialization, and the logits are multiplied by $m_d = (128/d)^{1/2}$.
3. *Hidden-layer reparametrization.* Each hidden matrix is written $W = m_d V$ with the same multiplier m_d, and V is initialized as W_{init}/m_d so that the function at initialization is unchanged. The optimizer updates V, and decoupled weight decay acts on V. The code comments state the intent: under Adam-type updates V moves at about the learning rate per step, so W moves at about m_d times the learning rate, which is meant to compensate for width in the effective step size.

The multiplier is $2.83, 2.0, 1.41, 1.0, 0.71$ at $d = 16, \ldots, 256$. The exponents (0.5) and the reference width (128) are fixed constants chosen before the final evaluation, not tuned on it. The training workload is dictated by the evaluator: AdamW with learning rate 10^{-3}, $\beta = (0.9, 0.98)$, decoupled weight decay 1.0, full-batch updates on all 3,763 training pairs, and 10,000 steps. We refer to this configuration as the *evaluated model*.

3.3 Reference trainer, controls, and validity

Reference trainer. The evaluator also trains its own stock transformer with the same optimizer settings and no modifications (no initialization shrinkage, no multipliers, all positions processed), using the first final seed (201) at each of the five widths. The OLS slope of $\ln \Delta$ on $\log_2 d$ over these five single-seed delays is the reference slope. It has no interval.

Generated by The Agentic Scientist

Table 1: Sealed final evaluation: aggregate metrics (four seeds $\times$ five widths $= 20$ runs, plus two controls). "Score" is the mean over seeds and widths of the area under the held-out-accuracy curve over the 40 checkpoints; it does not use the 0.99 threshold.

Quantity	Value
Slope of $\ln \Delta$ on $\log_2 d$ (point)	-0.020
95% seed-cluster bootstrap interval	$[-0.585,\ 0.354]$
Verdict under interval rule	inconclusive
Reference-trainer slope (seed 201 only, no interval)	-0.368
Censored fraction of the 20 runs	0.40 (8 runs)
Control maximum held-out accuracy (ceiling 0.10)	0.012
Missing episodes	0
Score (mean held-out-accuracy AUC)	0.729

Controls. Two control runs at width 64 and seed 201 check that the measurement does not manufacture generalization: one trains on training labels that have been permuted (at least 90% of labels changed) and one performs zero optimizer steps. A control fails if its held-out accuracy ever exceeds 0.10; chance level for 97 classes is about 0.010.

Validity. The evaluator marks the final evaluation valid (`qualified`, status `completed`) when, according to its source, all episodes completed, the compute and call budgets were respected, the controls passed, and at least one run fit the training data to 0.99. Only these three status fields are retained in the artifacts; per-condition diagnostics beyond them are not, so we treat any finer breakdown as unknown. Diagnostics reported by the evaluated model itself are untrusted and not used: every accuracy, delay, control, and statistic below is computed by the evaluator from returned predictions.

3.4 Scope of the retained evidence

The retained final artifacts consist of the aggregate metrics listed in Section 4 and one record per seed (mean validation-accuracy score and wall-clock time). *Per-width and per-seed delays, fitting and generalization steps, and per-width censoring flags are not retained*, so we cannot show a delay-versus-width plot, and we do not infer one. Separately, a development evaluation on a different data plane (modulus 53, training fraction 0.5, widths $32, 64, 128$, $6{,}000$ steps, two seeds) was used when configuring the evaluated model; its results are reported only as descriptive context in Appendix A.3 and never mixed with the sealed result.

4 Results

4.1 Main result: the slope test is inconclusive

The sealed evaluation completed with status `qualified`. Its aggregate metrics are listed in Table 1. The slope of $\ln \Delta$ on $\log_2 d$ is -0.020, with 95% seed-cluster bootstrap interval $[-0.585,\ 0.354]$. Because the interval contains zero, the interval rule returns *inconclusive*: the prediction that delay decreases with width is neither supported nor refuted. A point estimate of -0.020 corresponds to a fitted multiplicative change of 0.980 in delay per doubling of width, and the interval to factors between 0.557 and 1.425; over the full $16 \to 256$ range the fitted factors are 0.924 with interval $[0.096, 4.12]$. That is, the data are compatible with a delay roughly 10 times shorter at width 256 than at 16, with no change, and with a delay about 4 times longer.

4.2 Resolution of the design

An inconclusive verdict is informative only relative to the effects the design could have resolved. Three properties of the protocol, all computed from its constants and the reported interval (Table 2), limit that resolution.

Generated by The Agentic Scientist

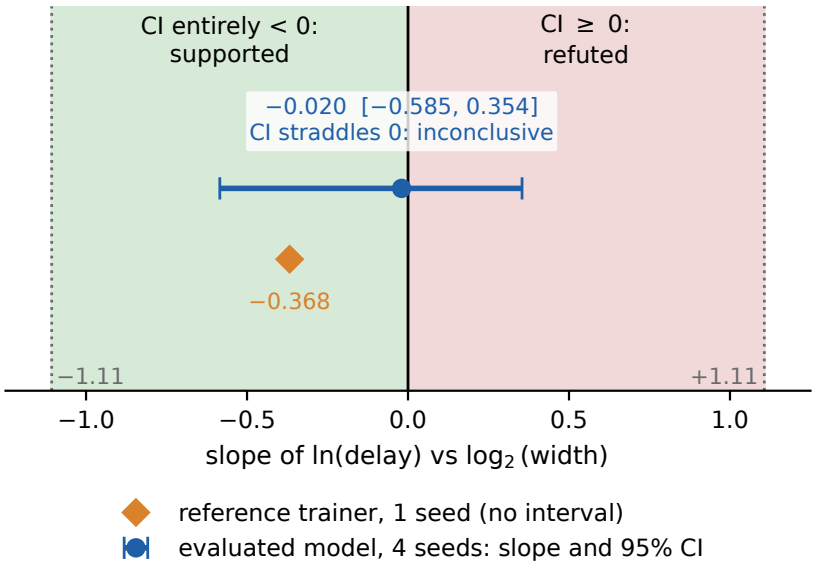


Figure 2: The estimated slope and its 95% interval against the three decision regions. The interval straddles zero, so the verdict is inconclusive. The single-seed reference slope lies inside the interval. Dotted lines mark the largest slope magnitude attainable (1.107) when every width-wise mean delay is confined to $[250, 10{,}000]$.

Bounded dynamic range. In this balanced design the slope is a fixed linear contrast of the width-wise mean log-delays $m_d = \frac{1}{4}\sum_s \ln \Delta_{d,s}$:

$$\beta = \sum_d w_d\, m_d, \qquad (w_{16}, \ldots, w_{256}) = (-0.2,\ -0.1,\ 0,\ 0.1,\ 0.2), \tag{3}$$

that is, $\beta = 0.2\,(m_{256} - m_{16}) + 0.1\,(m_{128} - m_{32})$; the middle width carries no weight, and the verdict depends only on how the two widest models compare with the two narrowest. Because every $m_d \in [\ln 250, \ln 10{,}000]$, $|\beta| \le 0.3 \ln 40 = 1.107$, attained by a step profile; a delay falling log-linearly from the cap at $d = 16$ to the floor at $d = 256$ would give 0.922. The interval has width 0.939 and covers 42% of the attainable range $[-1.107, 1.107]$. Its lower arm extends 0.565 below the point estimate and its upper arm 0.374 above.

Distance to a supported verdict. Keeping the upper arm of the interval unchanged, a verdict of supported would have required a point estimate below -0.374. The reference slope (-0.368) is slightly above that value; with the same upper arm it would give an upper endpoint of $+0.006$, still straddling zero. This is arithmetic on the observed spread, not a forecast: the spread of a different point estimate would differ.

A coarse bootstrap. With four seeds there are only $\binom{7}{4} = 35$ distinct multisets of seeds to resample, and a fraction $4/4^4 = 1/64$ of draws consist of a single seed repeated four times. The $2{,}000$ resamples therefore estimate a discrete distribution with at most 35 support points; the percentile endpoints -0.585 and 0.354 should be read as coarse, and the coverage of an interval built from four clusters is not guaranteed to be nominal.

4.3 Censoring

Forty percent of the 20 runs (8 runs) were censored at the step limit and entered the regression with the cap value $10{,}000$. Censoring has two consequences. First, because censored delays are replaced by the largest permitted value, any real dependence whose long-delay end is censored is compressed, so the fitted slope is biased toward zero when long delays concentrate at one end of the width range. Second, *censored* here covers two different events, held-out accuracy never reaching 0.99 and training accuracy never reaching 0.99; the retained artifacts do not say which occurred or

Generated by The Agent Scientist

Table 2: Quantities derived from the protocol constants and the reported interval (not additional measurements).

Quantity	Value
Delay floor / cap (steps)	250 / 10,000
Largest attainable |slope| (step profile)	1.107
Slope of a log-linear cap-to-floor profile	0.922
Interval width	0.939
Interval width as fraction of $[-1.107, 1.107]$	0.42
Interval arms below / above the point estimate	0.565 / 0.374
Per-doubling delay factor: point [interval]	0.980 [0.557, 1.425]
$16 \to 256$ delay factor: point [interval]	0.924 [0.096, 4.12]
$16 \to 256$ delay factor at the reference slope	0.230
Distinct bootstrap seed multisets	35

Table 3: Seed-level records from the sealed evaluation. Score: mean over the five widths of the held-out-accuracy AUC. Wall-clock time is summed over all episodes with that seed; seed 201 also includes the two control runs.

Seed	Score	Wall-clock (s)
201	0.607	2,468
202	0.733	1,894
203	0.717	2,062
204	0.860	1,943
Mean $\pm$ s.d.	0.729 ± 0.104	

at which widths, so they cannot separate delayed generalization from failure to fit. The score in Table 1 offers a threshold-free view: averaged over widths and seeds, held-out accuracy integrated over the 40 checkpoints is 0.729, so the runs on average generalize to a substantial degree within the window, but this aggregate does not locate where in the width range that happens.

4.4 Seed-level variability

The four seeds differ considerably in the score (Table 3), from 0.607 to 0.860 with mean 0.729 and sample standard deviation 0.104. We report this as variability across seeds, not as a performance comparison: each seed's score averages five widths, and a single seed supplies one run at every width. That the seed-to-seed spread is large relative to the quantities of interest is consistent with the wide interval for the slope, although the artifacts do not allow a variance decomposition across widths.

4.5 Cross-check against the reference trainer

The reference trainer, run on seed 201 only, gives a slope of -0.368, a fitted $16 \to 256$ delay factor of 0.230, and lies inside the interval, 23% of the way from its lower endpoint to its upper. The evaluated model's point estimate is 0.348 larger (less negative). We draw no conclusion about this difference: the reference slope is one seed with no uncertainty, five points define it, and the evaluated model differs from the reference in three simultaneous ways (Section 3.2). The comparison also does not establish that the multipliers weaken the width dependence: that would require both variants to be run with several seeds under the same protocol, which was not done.

4.6 Validity checks and development-plane context

The two controls stayed far below their ceiling (maximum held-out accuracy 0.012 against a ceiling of 0.10), no episodes were missing, and the evaluator reports a completed, valid evaluation. These checks support the integrity of the measurement. They do not bear on the width question, and a valid measurement can be inconclusive.

Generated by The Agentic Scientist

Table 4: What each reading of the interval would imply for quantities that are not retained in the artifacts. None of these can be checked with the available records.

Reading	Implication for per-width delays	Implication for censoring
(a) Little or no width dependence	Similar delays across widths, within seed noise	Censored runs not concentrated at either end
(b) Negative dependence masked by cap and noise	Longer delays at narrow widths, partly hidden at the cap	Censored runs concentrated at narrow widths
(c) Compensation by multipliers	Matched stock-model runs show a steeper slope than evaluated-model runs on the same seeds	Differs between the two models at the same widths
(d) Non-monotone in $\ln d$	Extremal delay at an intermediate width; contrast β near zero	Censoring at both ends or in the middle

For context, three configurations that differ cumulatively by the three modifications of Section 3.2 were evaluated on the separate development plane with two seeds each, with scores 0.798, 0.828, and 0.850 and 4, 6, and 4 of 6 runs censored (Appendix A.3). Differences between configuration means are no larger than the seed-to-seed spreads within a configuration, the plane differs from the sealed one in modulus, training fraction, widths, step limit, and seeds, and no slope interval was computed; we therefore draw no inference from them.

5 Discussion and Limitations

Relation to the explanations of grokking. Each account in Section 2 associates the delay with a quantity that can depend on width or parametrization: initial weight norm and the loss landscape, the relative efficiency of memorizing and generalizing circuits, and the position on the lazy-to-rich axis. The evaluated model touches the last two knobs directly: its initial weights are shrunk by half, which the source comments describe as aimed at reducing lazy-regime memorization, and its multipliers change how large a step each layer takes at a given width. A clear width slope from this model could therefore reflect the multipliers rather than width alone, and a flat slope could reflect their compensating a stock-model dependence. This is why the isolating comparison with a matched stock model (Section 5, item iv) matters more than a further estimate of the slope for the evaluated model alone.

Alternative readings that the data cannot separate. The interval is compatible with four readings (Table 4). (a) *Little or no width dependence* over 16–256 in this setting. (b) *A real negative dependence masked by censoring and noise*: cap-compressed delays and four seeds could produce a flat-looking slope. (c) *Compensation by the width-aware multipliers*: if the hidden and output multipliers equalize effective step sizes across width, a dependence present in the stock model could be reduced in the evaluated one; the single-seed reference slope (-0.368) is more negative than the point estimate, but it is one seed without an interval and does not by itself support this reading. (d) *Dependence that is non-monotone in* $\ln d$, which a linear slope cannot register. Each reading predicts something different about per-width delays and censoring flags, which are exactly the quantities not retained.

Threshold and time-grid sensitivity. The delay is read off a 250-step grid with a 0.99 threshold on both accuracies, so it is blind to differences below one probe interval and sensitive to where a run plateaus. The development evaluation shows how this can matter: all six runs of one configuration were censored even though their mean final held-out accuracy was 0.944 (Appendix A.3), and the retained records do not show whether that censoring came from a missed training threshold or a missed held-out threshold, or how close individual runs came to 0.99. We did not vary the threshold or the grid in the sealed evaluation, so we cannot say how the slope would change under, for example, a 0.95 threshold, and the retained artifacts do not allow it to be recomputed.

What would resolve it. Without claiming these were performed: (i) report per-width, per-seed delays with explicit censoring flags and the reason for censoring (not fitting vs. not generalizing);

Generated by The Agentic Scientist

Table 5: Claims and their status under this study's evidence.

Claim	Status	Basis
Delay decreases with width in this setting	Not established	Interval $[-0.585, 0.354]$ contains 0
Delay is independent of width in this setting	Not established	Same interval also contains slopes down to -0.585
The measurement was valid	Supported	Completed evaluation, controls 0.012 vs. ceiling 0.10, no missing episodes
A substantial share of runs is censored	Supported	8 of 20 runs
The multipliers weaken or remove a width dependence	Not tested	Reference slope has one seed, no interval, no matched ablation
Fitting time and generalization time scale differently with width	Not testable	Not retained in the artifacts
Findings carry over to other moduli, fractions, operations	Not tested	One configuration only

(ii) analyze delays as censored data instead of imputing the cap (survival-analysis regression, e.g. Cox, 1972, handles right-censored outcomes directly, although we did not apply it), or raise the step limit until few runs are censored; (iii) use more seeds, so that the bootstrap has more than 35 distinct resamples; (iv) run the multipliers' exponent at 0 and 0.5 under the same protocol to isolate their effect (no result for such a sweep exists in the retained artifacts); (v) vary the training fraction, the modulus, and the weight decay, which determine the delay in the accounts reviewed in Section 2; and (vi) record fitting and generalization steps separately to test the capacity-based reading directly.

Limitations. Table 5 summarizes which claims the evidence does and does not support. The evidence concerns one operation, one modulus (97), one training fraction (0.4), one optimizer setting, a single layer, and widths 16–256 (up to 0.84×10^6 parameters). The evaluated model contains three simultaneous modifications relative to the stock model, so even a clear slope would not be attributable to any one of them; sequential changes made during development support at most comparisons between whole configurations. The reference slope has one seed. The seed-cluster interval rests on four seeds. Censoring merges two events. The development-plane numbers come from a different workload with two seeds. We did not test the explanations in Section 2, did not run any weight-norm, circuit-level, or lazy-to-rich diagnostics, and did not run ablations of the three modifications. A valid evaluation does not imply scientific support for the prediction; the verdict is *inconclusive*, and the design's limits above, rather than a demonstrated absence of effect, are the most defensible reading.

6 Conclusion

We tested whether the grokking delay of a one-layer transformer on modular addition decreases with width, using a censoring-aware slope test with a seed-cluster bootstrap interval. The sealed evaluation was valid and gave a slope of -0.020 with interval $[-0.585, 0.354]$ over widths 16–256 and four seeds, so the prediction is neither supported nor refuted. Forty percent of runs were censored at the step limit, the interval covers about two-fifths of the slopes that the delay floor and cap allow, and a four-cluster bootstrap has only 35 distinct resamples. A single-seed reference trainer gave -0.368, inside the interval. We therefore do not claim that delay depends on width, nor that it does not, and we do not attribute the difference between the reference and the evaluated model to any of its modifications. The practical lesson is a measurement one: for a delay statistic with a floor and a censoring cap, report per-width delays and censoring flags, use enough seeds that the cluster bootstrap has a rich resampling support, and treat the attainable slope range as part of the result. The per-width records, the isolating ablation of the multipliers, and sweeps over training fraction and weight decay would turn this inconclusive result into a testable one.

Generated by Some Agentic Scientist

REFERENCES

A. Cameron and Douglas L. Miller. A practitioner's guide to cluster-robust inference. *The Journal of Human Resources*, 50:317 – 372, 2015.

A. Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90:414–427, 2008.

D. Cox. Regression models and life-tables. *Journal of the royal statistical society series b-methodological*, 34:187–220, 1972.

Xander Davies, Lauro Langosco, and David Krueger. Unifying grokking and double descent. *ArXiv*, abs/2303.06173, 2023.

Darshil Doshi, Aritra Das, Tian-Yu He, and Andrey Gromov. To grok or not to grok: Disentangling generalization and memorization on corrupted algorithmic datasets. *ArXiv*, abs/2310.13061, 2023.

Yu-Fei Huang, Sheng-Ding Hu, Xu Han, Zhi-Yuan Liu, and Mao-Song Sun. Unified view of grokking, double descent and emergent abilities: A perspective from circuits competition. *ArXiv*, abs/2402.15175, 2024.

Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics. *ArXiv*, abs/2310.06110, 2023.

Zi-Ming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. *ArXiv*, abs/2210.01117, 2022.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ArXiv*, abs/1711.05101, 2017.

Clare Lyle, Ze-Yu Zheng, Khimya Khetarpal, James Martens, H. V. Hasselt, Razvan Pascanu, and Will Dabney. Normalization and effective learning rates in reinforcement learning. *ArXiv*, abs/2407.01800, 2024.

John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. How much do language models memorize? *ArXiv*, abs/2505.24832, 2025.

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *ArXiv*, abs/1912.02292, 2020.

Yoonsoo Nam, Nayara Fonseca, S. Lee, Chris Mingard, N. Göring, Ouns El Harzli, Abdurrahman Hadi Erturk, Soufiane Hayou, and Ard A. Louis. Decoupling dynamical richness from representation learning: Towards practical measurement. 2024.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jesse Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. *ArXiv*, abs/2301.05217, 2023.

Alethea Power, Yuri Burda, Harrison Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *ArXiv*, abs/2201.02177, 2022.

Lucas Prieto, Melih Barsbey, P. Mediano, and Tolga Birdal. Grokking at the edge of numerical stability. *ArXiv*, abs/2501.04697, 2025.

Vimal Thilak, Etai Littwin, Shuang-Fei Zhai, O. Saremi, Roni Paiss, and J. Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *ArXiv*, abs/2206.04817, 2022.

Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. *ArXiv*, abs/2205.10770, 2022.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *ArXiv*, abs/2309.02390, 2023.

Generated by The Agentic Scientist

Greg Yang and J. Hu. Tensor programs iv: Feature learning in infinite-width neural networks. pp. 11727–11737, 2021.

Greg Yang, J. Hu, Igor Babuschkin, S. Sidor, Xiaodong Liu, David Farhi, N. Ryder, J. Pachocki, Wei-Zhu Chen, and Jian-Feng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *ArXiv*, abs/2203.03466, 2022.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *ArXiv*, abs/1611.03530, 2017.

Generated by The Agentic Scientist

A Supplementary Information

A.1 Protocol and hyperparameters

Table 6 lists the workloads of the sealed evaluation and of the development evaluation, as specified in the evaluator's workload manifests (`source_evidence/workload/`).

Table 6: Workloads. The evaluator owns all items; the evaluated model receives only the training rows.

	Sealed final evaluation	Development evaluation
Operation, modulus p	addition, 97	addition, 53
Training fraction	0.4 (3,763 pairs)	0.5
Widths d	$16, 32, 64, 128, 256$	$32, 64, 128$
Seeds	201–204	$101, 102$
Step limit / probe interval	10,000 / 250	6,000 / 300
Control width	64	64
Optimizer	AdamW, lr 10^{-3}, $\beta = (0.9, 0.98)$, weight decay 1.0, full batch	
Heads, layers	4, 1	
Fit / generalization threshold	0.99 accuracy	
Bootstrap (final only)	2,000 seed-cluster resamples	–

The final evaluation made $1{,}804$ calls to the evaluated model (limit $2{,}200$; 22 episodes, each a reset, a data load, and 40 train/predict pairs) and took $11{,}764$ s of wall-clock time including the reference trainer.

A.2 Evaluated model: constants and parameter counts

Table 7 gives the width-dependent constants and parameter counts of the evaluated model. The parameter count follows from the layer shapes in the source: embeddings $(p+1)d$, positional embedding $3d$, attention projections $3d^2 + d^2$ (no bias), MLP $4d^2 + 4d$ and $4d^2 + d$, unembedding pd (no bias), totalling $12d^2 + (2p+9)d$. The initial-weight scale (0.5), the unembedding scale (0.5), the reference width (128), and both exponents (0.5) are constants in the source; the code comments state they were chosen without offline tuning.

Table 7: Parameter count and multiplier $m_d = (128/d)^{1/2}$ by width.

Width d	Parameters	m_d
16	6,320	2.83
32	18,784	2.00
64	62,144	1.41
128	222,592	1.00
256	838,400	0.71

A.3 Development-plane comparison (descriptive, not final)

Figure 3 shows the three development configurations: V1 (smaller initial weights and last-position-only computation), V2 (V1 plus width-aware output scaling), and V3 (V2 plus hidden-layer reparametrization; the evaluated model). Scores are means over two seeds on the development plane; per-seed scores are plotted. Table 8 adds the mean final held-out accuracy and the censoring counts. The rank order by score is reversed relative to the end-of-training accuracy: the mean final held-out accuracy decreases from V1 to V3 (0.981, 0.944, 0.921) while the score increases (0.798, 0.828, 0.850), so the score and the end-of-training accuracy measure different things. All six V2 runs are censored although their mean final held-out accuracy is 0.944; the retained records do not show whether the censoring came from a missed training threshold or a missed held-out threshold, or how close individual runs came to 0.99. The lower V3 seed (0.801) lies just above the V1 mean (0.798), while the higher V3 seed (0.899) lies above the V2 mean (0.828). With two seeds

Generated by the Agentic Scientist

per configuration, the seed-to-seed spreads (0.051, 0.043, 0.098) are no smaller than the differences between configuration means (0.031, 0.022), so no ordering of the configurations is supported.

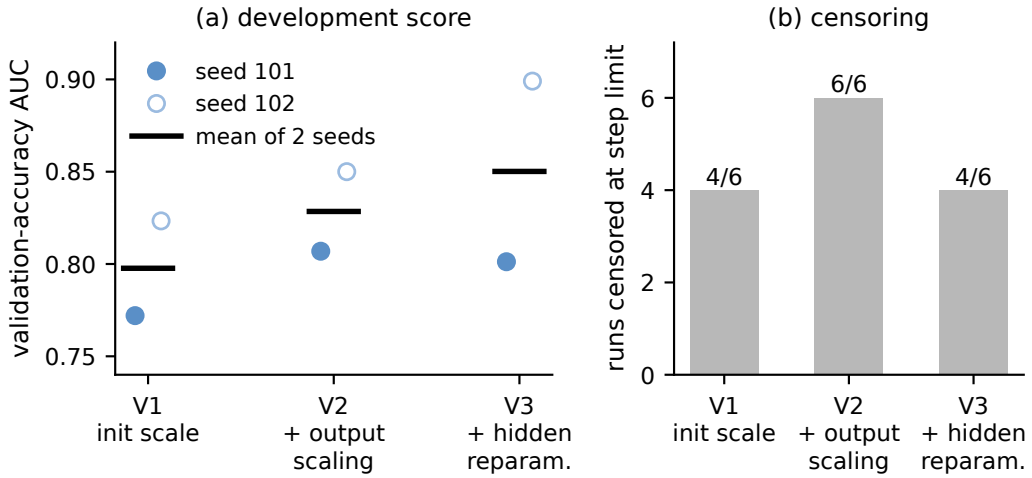


Figure 3: Development-plane comparison of three configurations (modulus 53, training fraction 0.5, two seeds; not the sealed evaluation). (a) Per-seed and mean held-out-accuracy AUC. (b) Number of the six runs censored at the step limit.

Table 8: Development-plane summary (two seeds $\times$ three widths $=$ 6 runs per configuration).

Configuration	Score	Mean final held-out acc.	Censored runs	Control max acc.
V1	0.798	0.981	4/6	0.023
V2	0.828	0.944	6/6	0.022
V3 (evaluated)	0.850	0.921	4/6	0.023

A.4 Analyses and hypotheses not performed

The following were considered during development and have *no* results in the artifacts: a capacity-crossover account in which fitting time falls faster with width than generalization time (separate fitting and generalization steps are not retained); a critical width as a function of training fraction; a sweep of the hidden-layer exponent; a sweep of the output-scaling exponent and unembedding scale; an intervention on weight decay; and any weight-norm or circuit-level diagnostics. Nothing in the main text depends on them.

A.5 Reproducibility

The accompanying files are: the evaluated model's source and the two evaluators (`source_evidence/`); the aggregate and per-seed final records (`source_evidence/final/`); `scripts/derive_resolution.py`, which computes every derived number in Tables 2 and 7 and the text from the evaluator records and writes `derived/resolution.json`; and `scripts/build_figures.py`, which produces Figures 2 and 3 from that file. Figure 1 is a schematic with no data. Data are generated in memory from the modulus and seed, with no download. The aggregate metrics (slope, interval, reference slope, censored fraction, control, score) are those recorded by the evaluator. Re-running the evaluator requires the evaluated model's dependencies (PyTorch and NumPy); the sealed evaluation took about 3.3 hours of wall-clock time on the recorded hardware, whose specification is not retained in the artifacts.

Generated by The Agentist

A.6 LLM USAGE

Large language models acted as autonomous agents in this work: they proposed the candidate directions, wrote and modified the evaluated model's code, drafted the analysis scripts, and wrote this manuscript. The task environment supplied the evaluators and the workload manifests; the models did not edit them. The interval rule that maps the evaluator's reported interval to a verdict was applied by our own script `scripts/derive_resolution.py`. No results were produced by an LLM other than by executing the code described above, and all numbers in the paper come from the evaluator records or from the derivations in `scripts/derive_resolution.py`.

Generated by The Agentic Scientist